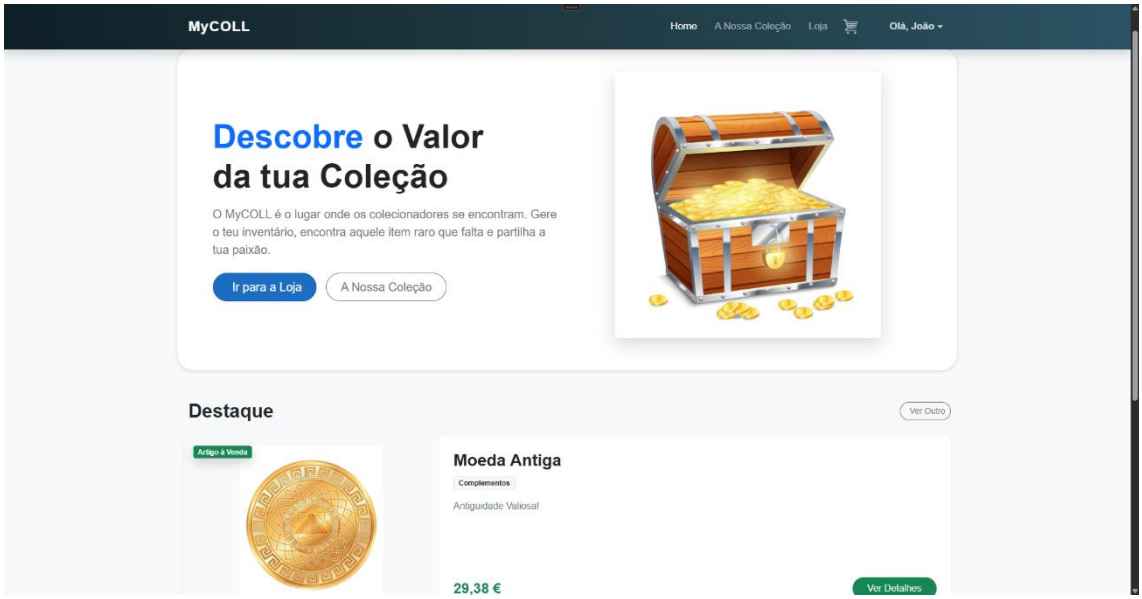
Instituto Superior de Engenharia de Coimbra
Instituto Politécnico de Coimbra

Relatório – Trabalho Prático de Programação WEB

2025/2026

Licenciatura em Engenharia Informática



Trabalho realizado por:

Rodrigo Glória -2022150127

Martim Freitas – 2021145193

Índice

1. INTRODUÇÃO E ENQUADRAMENTO TEÓRICO 4
1.1. Arquitetura Client-Server e API RESTful 4
1.2. O Paradigma Blazor Hybrid e a Reutilização de Código 4
1.3. Persistência de Dados e ORM 4
2. OBJETIVOS DO PROJETO 5
2.1. Aplicação Frontend (Interface Pública) 5
2.2. Aplicação de Gestão (Backoffice) 5
2.3. Objetivos Técnicos e Arquiteturais 6
3. ARQUITETURA E ESTRUTURA DO PROJETO 6
3.1. Visão Geral da Arquitetura 6
3.2. Organização da Solução (Projetos) 7
3.3. Bibliotecas de Classes Partilhadas (RCL) 7
4. MODELO DE DADOS E PERSISTÊNCIA 8
4.1. Diagrama Entidade-Relacionamento (MER) 8
4.2. Dicionário de Dados 8
5. REGRAS DE NEGÓCIO 10
5.1. Gestão de Acessos e Perfis (Roles) 10
5.2. Gestão de Produtos e Validação 10
5.3. Processamento de Encomendas 11
5.4. Carrinho de Compras Persistente 11
6. FUNCIONALIDADES EXTRA 11
6.1. Gestão de Favoritos (Wishlist) 11
6.2. Área Dedicada ao Fornecedor 12
6.3. Inicialização Automática (Seeding) 12
7. CREDENCIAIS DE ACESSO 12
7.1. Administrador (Backoffice e App) 12
7.2. Outros Perfis 12
8. PRINCIPAIS DIFICULDADES E ESTRATÉGIAS DE SUPERAÇÃO 12
8.1. Sincronização de Autenticação (Mobile vs Web) 13
8.2. Partilha de Recursos Estáticos (Imagens) 13
8.3. Conetividade e CORS em Desenvolvimento 13
9. ANÁLISE AUTOCRÍTICA E CONCLUSÃO 14
9.1. Análise dos Resultados 14
9.2. Conclusão 14

1. INTRODUÇÃO E ENQUADRAMENTO TEÓRICO

O presente relatório documenta o processo de conceção e desenvolvimento do projeto MyCOLL, uma solução de software para a gestão de colecionismo, realizada no âmbito da Unidade Curricular de Programação Web. O projeto visa a implementação de um sistema robusto capaz de responder aos desafios modernos da computação móvel e web, nomeadamente a necessidade de consistência de dados entre múltiplas plataformas.

No contexto atual da Engenharia de Software, a transição de arquiteturas monolíticas para Sistemas Distribuídos tornou-se um padrão da indústria. Este projeto reflete essa mudança de paradigma, adotando uma arquitetura onde a lógica de negócio e o armazenamento de dados são desacoplados da interface do utilizador. O sistema foi integralmente desenvolvido sobre a plataforma .NET 8, tirando partido da sua capacidade de unificar o desenvolvimento *backend* e *frontend* sob uma linguagem comum (C#).

O trabalho prático assenta em três pilares teóricos fundamentais que suportam a implementação técnica:

1.1. Arquitetura Client-Server e API RESTful

Ao contrário das aplicações web tradicionais que renderizam todo o conteúdo no servidor, o MyCOLL adota uma abordagem baseada em serviços. O núcleo do sistema é uma Web API (Application Programming Interface) que segue os princípios da arquitetura REST (Representational State Transfer).

- Teoria: A API atua como o servidor centralizado, expondo recursos (Produtos, Encomendas) através de operações HTTP padronizadas. Esta abordagem garante a interoperabilidade, permitindo que diferentes clientes (neste caso, uma aplicação móvel MAUI e um website Blazor) consumam os mesmos dados de forma agnóstica à sua implementação interna.

1.2. O Paradigma Blazor Hybrid e a Reutilização de Código

Para a componente cliente (Mobile/Desktop), foi adotada a tecnologia .NET MAUI com Blazor Hybrid. Esta escolha representa uma evolução face ao desenvolvimento nativo tradicional.

- Teoria: O Blazor Hybrid funde a capacidade de execução de código C# nativo (.NET runtime) com a renderização de interfaces web (HTML/CSS) através de um *WebView*. Teoricamente, isto permite a aplicação do princípio DRY (Don't Repeat Yourself) ao nível da interface, possibilitando que componentes visuais e lógica de validação sejam partilhados entre ambientes Web e Mobile sem duplicação de esforço ou inconsistências.

1.3. Persistência de Dados e ORM

A gestão de dados é suportada pelo SQL Server. Para fazer a ponte entre o paradigma Orientado a Objetos (da linguagem C#) e o paradigma Relacional (da base de dados), utiliza-se o Entity Framework Core.

- Teoria: O EF Core atua como um ORM (Object-Relational Mapper), abstraindo a complexidade das consultas SQL. Isto permite manipular os dados como objetos

fortemente tipados, garantindo a integridade referencial e facilitando a manutenção do esquema de dados através de migrações (*Code-First approach*).

Em suma, o projeto MyCOLL não é apenas uma ferramenta funcional de comércio eletrónico, mas sim um exercício prático de integração de arquiteturas modernas, demonstrando como a separação de responsabilidades (SoC - *Separation of Concerns*) e a abstração de camadas contribuem para um software escalável e de fácil manutenção.

2. OBJETIVOS DO PROJETO

O objetivo primordial deste projeto consistiu na conceção e desenvolvimento do sistema MyCOLL, uma plataforma integrada para a gestão de colecionismo. O sistema foi desenhado para operar num ambiente distribuído, permitindo a interação de diferentes tipos de utilizadores (Administradores, Funcionários, Fornecedores e Clientes) através de interfaces adaptadas às suas necessidades específicas.

Os objetivos foram estruturados em três vertentes principais:

2.1. Aplicação Frontend (Interface Pública)

A componente cliente, destinada ao público em geral, foi desenvolvida com o objetivo de oferecer uma experiência de utilização fluida e multiplataforma.

- Acessibilidade Universal: Garantir o acesso ao catálogo de produtos através de dispositivos móveis (Android/iOS) e Desktop (Windows/Mac) utilizando uma única base de código .NET MAUI Blazor Hybrid.
- Experiência de Comércio: Implementar funcionalidades essenciais de e-commerce, incluindo pesquisa de produtos por categorias (ex: Moedas, Selos), gestão de um Carrinho de Compras persistente e processamento de encomendas.
- Envolvimento do Utilizador: Possibilitar a criação de listas de Favoritos e permitir que utilizadores externos (Fornecedores) submetam os seus próprios produtos para venda na plataforma.

2.2. Aplicação de Gestão (Backoffice)

Para a administração do sistema, o objetivo foi criar uma ferramenta web robusta, focada na eficiência operacional.

- Centralização Administrativa: Desenvolver um painel em Blazor Server que permita aos gestores controlar todo o ecossistema (Produtos, Utilizadores e Encomendas) em tempo real.
- Controlo de Qualidade: Implementar fluxos de validação, onde produtos submetidos por fornecedores requerem aprovação explícita antes de ficarem visíveis ao público.
- Gestão de Entidades: Permitir a criação dinâmica de Categorias e Modos de Entrega, garantindo que a plataforma se adapta a novos tipos de colecionáveis sem necessidade de alteração de código.

2.3. Objetivos Técnicos e Arquiteturais

Transversalmente, o projeto visou cumprir requisitos técnicos rigorosos que asseguram a qualidade do software:

- Arquitetura Orientada a Serviços: Desacoplar a lógica de negócio das interfaces gráficas através de uma API RESTful, garantindo segurança e escalabilidade.
- Persistência e Integridade: Assegurar que todos os dados são armazenados de forma relacional e consistente numa base de dados SQL Server, utilizando Entity Framework Core para abstração de dados.
- Reutilização de Código: Aplicar o princípio *DRY (Don't Repeat Yourself)* através de bibliotecas de classes partilhadas (RCL), unificando a lógica de validação e os modelos de dados (DTOs) entre todas as aplicações da solução.

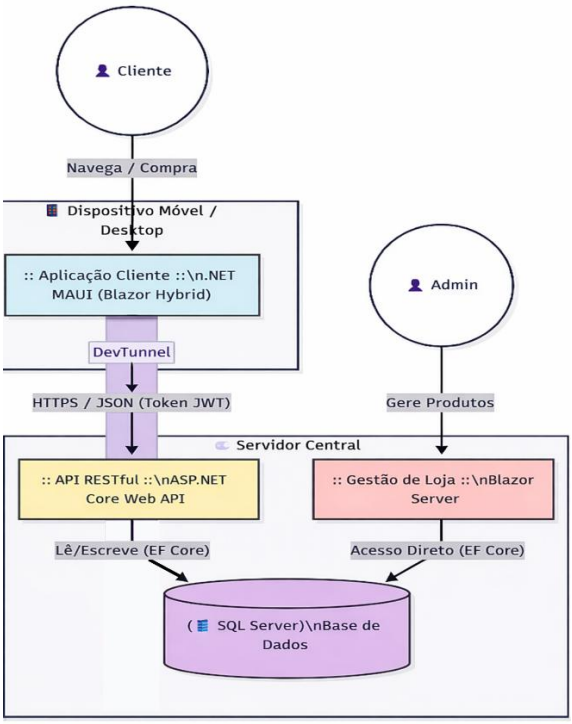
3. ARQUITETURA E ESTRUTURA DO PROJETO

A arquitetura do projeto MyCOLL segue o padrão de Sistemas Distribuídos Multicamada, concebida para maximizar a separação de responsabilidades e a reutilização de código. A solução foi estruturada de forma modular, permitindo que a lógica de negócio, a interface de utilizador e o acesso a dados evoluam de forma independente.

3.1. Visão Geral da Arquitetura

O sistema é composto por três blocos lógicos principais que comunicam entre si:

1. Camada de Dados: Suportada pelo SQL Server, onde reside toda a informação persistente.
2. Camada de Serviços (Backend): Uma API RESTful que centraliza as regras de negócio e a segurança, servindo como único ponto de entrada para a aplicação móvel.
3. Camada de Apresentação (Frontend): Dividida em dois clientes distintos:
    - Aplicação Móvel (ClienteMAUI): Uma aplicação híbrida que corre no dispositivo do utilizador, consumindo a API.
    - Aplicação Web (GestãoLoja): Uma aplicação de servidor que acede diretamente à base de dados para operações administrativas de alto desempenho.



3.2. Organização da Solução (Projetos)

A solução Visual Studio encontra-se dividida em projetos específicos, cada um com uma responsabilidade clara:

- API (Backend): Projeto ASP.NET Core Web API. Contém os *Controllers* (pontos de acesso), a configuração do *Swagger* e a camada de acesso a dados (*Repositories*).
- ClienteMAUI (Mobile): Projeto .NET MAUI Blazor Hybrid. Atua como um contentor nativo que renderiza os componentes web num *BlazorWebView*, permitindo a execução em Android, iOS, Windows e Mac.
- GestaoLoja (Web): Projeto Blazor Server. Responsável pelo Backoffice, tirando partido da renderização no servidor para uma gestão rápida de dados.

3.3. Bibliotecas de Classes Partilhadas (RCL)

Uma das principais inovações arquiteturais deste projeto é o uso extensivo de Razor Class Libraries (RCL). Para evitar a duplicação de código entre a aplicação Web e Mobile, a lógica e a interface foram extraídas para bibliotecas reutilizáveis:

- RCLComum: Contém componentes visuais transversais, como o *Layout* principal, o Menu de Navegação e as páginas de Autenticação (*Login*, *Registo*).
- RCLAPI: Encapsula toda a lógica de comunicação HTTP e os serviços de cliente (ex: CarrinhoService, AuthService). Contém também os DTOs (*Data Transfer Objects*) partilhados, garantindo que o Frontend e o Backend "falam a mesma língua".
- RCLProdutos: Centraliza as páginas e componentes relacionados com o catálogo, como a listagem de produtos e detalhes, permitindo que estas interfaces sejam invocadas por qualquer projeto da solução.

- RCLImages: Centraliza os recursos estáticos (imagens e ícones), resolvendo o problema de caminhos relativos em diferentes plataformas.

Esta estrutura modular não só facilita a manutenção, como demonstra uma aplicação prática dos princípios SOLID e da arquitetura limpa.

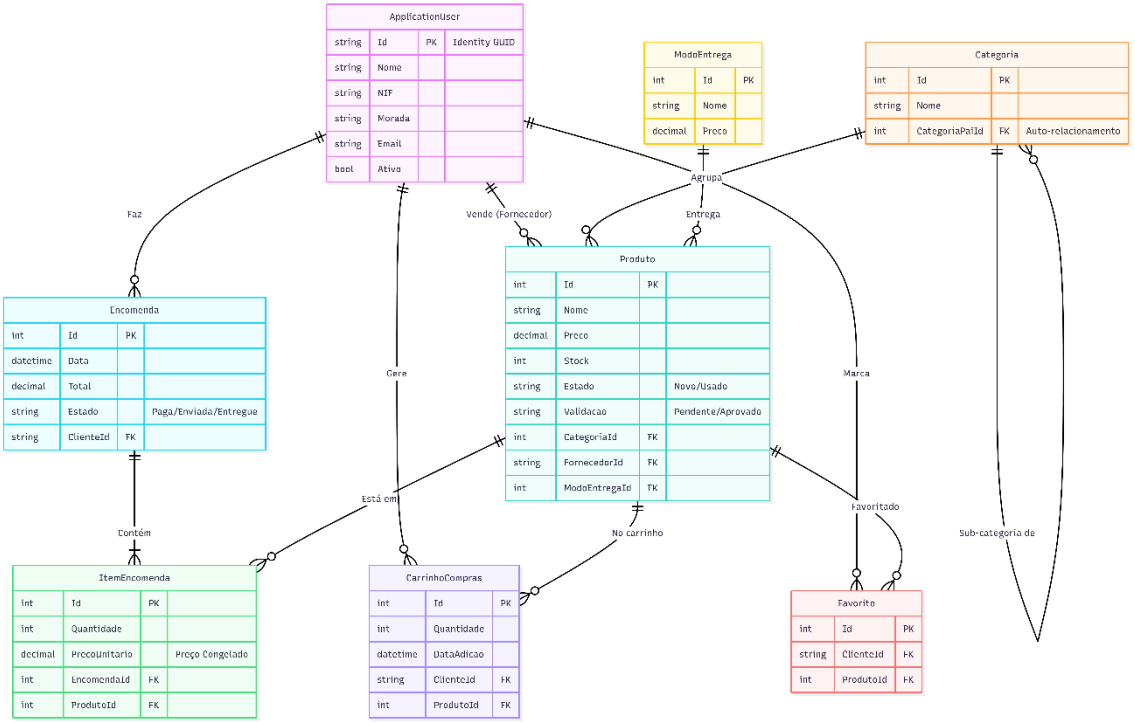
4. MODELO DE DADOS E PERSISTÊNCIA

A integridade e persistência da informação no sistema MyCOLL são asseguradas por uma base de dados relacional Microsoft SQL Server. A interação com a base de dados não é feita através de comandos SQL diretos, mas sim utilizando o Entity Framework Core (EF Core), um ORM (Object-Relational Mapper) que permite manipular os dados como objetos C# fortemente tipados.

Foi adotada a abordagem Code-First, onde as classes do código definem a estrutura da base de dados. As alterações ao esquema (ex: adicionar um campo "Preço") são geridas através de Migrações (Migrations), o que permite versionar a base de dados juntamente com o código da aplicação.

4.1. Diagrama Entidade-Relacionamento (MER)

O esquema relacional foi desenhado para suportar as operações de e-commerce e gestão de inventário. O diagrama abaixo ilustra as principais entidades e as suas relações.



4.2. Dicionário de Dados

Abaixo descrevem-se as principais tabelas mapeadas no contexto de dados (ApplicationDbContext):

1. Tabela: ApplicationUser (Utilizadores)

Estende a tabela padrão AspNetUsers do sistema de identidade. Armazena todos os intervenientes no sistema (Administradores, Clientes e Fornecedores).

- Campos Adicionais: Nome, NIF, Morada, Estado (Ativo/Pendente).
- Relações:
    - 1:N com Encomenda (Um cliente faz várias encomendas).
    - 1:N com Produto (Um fornecedor pode submeter vários produtos).

2. Tabela: Produto

A entidade central do sistema. Representa o item colecionável disponível para venda ou exposição.

- Campos: Nome, Descricao, Preco, ImagemUrl, EstadoProduto (Novo/Usado), EstadoValidacao (Pendente/Aprovado).
- Relações:
    - N:1 com Categoria (Cada produto pertence a uma categoria).
    - N:1 com ModoEntrega (Define como o produto pode ser enviado).
    - N:1 com ApplicationUser (O fornecedor que possui o item).

3. Tabela: Categoria

Estrutura hierárquica que organiza o catálogo.

- Campos: Nome, Descricao.
- Relações:
    - Auto-Relacionamento (1:N): Uma categoria pode ter uma "Categoria Pai" (ex: "Moedas" é pai de "Euros"). Isto permite criar uma árvore de categorias de profundidade infinita.

4. Tabela: Encomenda

Regista o histórico de transações finalizadas.

- Campos: Data, ValorTotal, Estado (Pendente, Paga, Enviada, Entregue), MoradaEnvio.
- Relações:
    - N:1 com ApplicationUser (Cliente).
    - 1:N com ItemEncomenda (Detalhes dos produtos comprados, guardando o preço no momento da compra).

5. Tabela: CarrinhoCompras

Permite a persistência da intenção de compra. Diferente de uma sessão temporária, esta tabela guarda os itens na base de dados, permitindo que o utilizador inicie a compra no telemóvel e a termine noutro dispositivo.

- Relações: Liga um Produto a um ApplicationUser.

6. Tabela: Favorito

Implementa a funcionalidade de "Wishlist".

- Relações: Tabela de ligação simples entre ApplicationUser e Produto.

5. REGRAS DE NEGÓCIO

As regras de negócio definem a lógica operacional do sistema MyCOLL, garantindo a integridade dos dados e o correto fluxo de processos entre os diferentes atores. Estas regras foram implementadas tanto ao nível da API (para segurança) como da Interface (para usabilidade).

5.1. Gestão de Acessos e Perfis (Roles)

O sistema implementa um controlo de acesso baseado em funções (*Role-Based Access Control* - RBAC). Existem quatro perfis distintos, definidos na estrutura de dados:

1. Administrador: Possui controlo total sobre a plataforma. Pode gerir utilizadores, atribuir cargos a funcionários, e tem acesso irrestrito a todas as funcionalidades de gestão de produtos e encomendas.
2. Funcionário: Atua como um operador de Backoffice. Tem permissão para validar produtos e processar encomendas, mas não pode gerir outros utilizadores administrativos.
3. Fornecedor: Um utilizador externo que pode submeter produtos para venda. Tem acesso a uma área reservada na App para consultar o estado das suas submissões e o histórico de vendas.
4. Cliente: O utilizador final padrão. Pode navegar no catálogo, gerir o carrinho de compras, adicionar favoritos e realizar encomendas.

Regra de Registo: Novos utilizadores que se registam pela aplicação móvel são automaticamente atribuídos à role "Cliente". A promoção a "Fornecedor" ou "Funcionário" requer intervenção administrativa no Backoffice.

5.2. Gestão de Produtos e Validação

Para garantir a qualidade do catálogo, foi implementado um fluxo de aprovação para produtos submetidos por terceiros.

- Submissão: Quando um Fornecedor cria um produto, este entra no sistema com o estado Pendente e não é visível imediatamente no catálogo público.
- Validação: Um Administrador ou Funcionário deve analisar o produto no Backoffice. Apenas após a alteração do estado para Aprovado é que o item fica disponível para compra na API.
- Visibilidade: Apenas os produtos marcados explicitamente como "Inativos" ou “Rejeitados” pela gestão de loja são ocultados do catálogo público. Produtos com stock igual a zero permanecem visíveis nas listagens de clientes até que a administração

oculte, sendo apresentados com a indicação de "Esgotado", permitindo assim informar o cliente sobre a inexistência de stock do artigo."

5.3. Processamento de Encomendas

O ciclo de vida de uma encomenda segue uma máquina de estados estrita para evitar inconsistências financeiras ou de stock:

1. Criação (Pendente): A encomenda é registada, o stock dos produtos é provisoriamente cativado, mas o envio não é autorizado.
2. Pagamento (Paga): Após confirmação do pagamento (simulado no sistema), o estado transita para Paga.
3. Expedição (Enviada): O gestor de loja marca a encomenda como enviada, inserindo opcionalmente dados de rastreio.
4. Aguarda receção: Aqui o gestor aguarda que o cliente informe a receção da encomenda.
5. Entrega e Conclusão: Estado final que encerra o processo.

Regra de Stock: O sistema impede a finalização de uma encomenda se a quantidade solicitada de qualquer item exceder o stock disponível no momento do *checkout*.

5.4. Carrinho de Compras Persistente

Ao contrário de sessões voláteis, o carrinho de compras é guardado na base de dados.

- Regra de Sincronização: Se um utilizador adicionar itens como anónimo e posteriormente fizer *login*, o sistema associa os itens do carrinho temporário à conta do utilizador, garantindo que não se perdem intenções de compra. Assim que o cliente faça *logout*, o carrinho é limpo.

6. FUNCIONALIDADES EXTRA

Para além dos requisitos base estipulados no enunciado, foram implementadas funcionalidades adicionais que visam enriquecer a experiência de utilização e aproximar o projeto de um cenário real de produção.

6.1. Gestão de Favoritos (Wishlist)

Foi implementado um sistema completo de gestão de favoritos, permitindo aos utilizadores autenticados marcar produtos de interesse sem necessidade de os adicionar imediatamente ao carrinho.

- Implementação: A funcionalidade é suportada pela entidade Favorito na base de dados e pelo FavoritosController na API.
- Interface: Na aplicação móvel, os utilizadores podem visualizar os seus itens guardados numa página dedicada e alternar o estado do favorito (ícone de coração) diretamente no catálogo.

6.2. Área Dedicada ao Fornecedor

Para além do perfil de Cliente, foi desenvolvida uma área específica para Fornecedores na aplicação móvel.

- Funcionalidade: Os fornecedores têm acesso a um *dashboard* onde podem consultar o estado dos produtos que submeteram ("Pendente", "Aprovado", "Rejeitado") e visualizar um histórico das suas vendas concretizadas.
- Valor: Isto descentraliza a introdução de dados, permitindo que a comunidade alimente o catálogo, enquanto a administração apenas modera o conteúdo.

6.3. Inicialização Automática (Seeding)

Para facilitar a instalação e avaliação do projeto, foi criado um mecanismo de *Seeding* robusto.

- Automatismo: Ao arrancar o Backoffice pela primeira vez, o sistema verifica a inexistência de dados e cria automaticamente as Roles necessárias, o Utilizador Administrador e as Categorias base do negócio (Moedas, Selos, etc.). Isto elimina a necessidade de executar scripts SQL manuais para configurar o ambiente.

7. CREDENCIAIS DE ACESSO

Para efeitos de teste e avaliação da plataforma, o sistema encontra-se pré-configurado com uma conta de administração principal.

7.1. Administrador (Backoffice e App)

Esta conta possui permissões totais ("Super Admin"), permitindo aceder tanto à gestão de loja como à aplicação cliente.

- Email: admin@mycoll.pt
- Password: Admin123!

7.2. Outros Perfis

Para testar os perfis de Cliente ou Fornecedor, recomenda-se a utilização da funcionalidade de "Registo" na aplicação móvel:

1. Registar uma nova conta na App (ficará com perfil Cliente por defeito).
2. Para testar o perfil Fornecedor, aceder ao Backoffice com a conta de Admin e alterar a Role do novo utilizador para "Fornecedor".

8. PRINCIPAIS DIFICULDADES E ESTRATÉGIAS DE SUPERAÇÃO

O desenvolvimento de uma solução distribuída que integra tecnologias Web, Mobile e API envolveu desafios técnicos significativos. Abaixo descrevem-se os principais obstáculos encontrados e as soluções adotadas.

8.1. Sincronização de Autenticação (Mobile vs Web)

- Dificuldade: Um dos maiores desafios foi gerir a autenticação em dois ambientes distintos. Enquanto o Backoffice (Blazor Server) utiliza Cookies de sessão geridos pelo ASP.NET Identity, a Aplicação Móvel (Blazor Hybrid) não consegue manter cookies de forma persistente e segura entre reinícios da aplicação, necessitando de uma abordagem baseada em Tokens (JWT).
- Estratégia: A solução passou pela implementação de um sistema híbrido. A API foi configurada para emitir e validar JWTs. No lado do cliente móvel, foi criado um AuthService personalizado que interceta o login, armazena o token no armazenamento seguro do dispositivo (LocalStorage) e injeta-o automaticamente no cabeçalho de todos os pedidos HTTP subsequentes.

8.2. Partilha de Recursos Estáticos (Imagens)

- Dificuldade: Referenciar imagens (logótipos, ícones de produtos) de forma consistente entre a App (que corre no telemóvel) e o Site (que corre no servidor) revelou-se problemático, pois os caminhos relativos (~/images/) não funcionam da mesma forma em ambas as plataformas.
- Estratégia: Foi criada uma biblioteca específica, a RCLImages, que centraliza todos os recursos estáticos. Desta forma, ambos os projetos referenciam os ficheiros como recursos embebidos ou através de caminhos virtuais estáticos (_content/RCLImages/...), garantindo que as imagens carregam corretamente em qualquer ambiente.

8.3. Conetividade e CORS em Desenvolvimento

- Dificuldade: Durante os testes, a aplicação móvel (a correr no emulador Android) era frequentemente bloqueada ao tentar aceder à API local devido a políticas de segurança do browser (CORS).
- Estratégia: Foi configurada uma política de CORS permissiva ("AllowAll") no ambiente de desenvolvimento da API, permitindo pedidos de qualquer origem, facilitando assim a comunicação entre o emulador e o servidor local.

9. ANÁLISE AUTOCRÍTICA E CONCLUSÃO

O projeto MyCOLL atingiu com sucesso os objetivos propostos, resultando numa plataforma funcional de gestão de colecionismo que cumpre os requisitos de um sistema distribuído moderno.

9.1. Análise dos Resultados

A arquitetura baseada em .NET 8 e Blazor Hybrid provou ser uma escolha acertada. A utilização de Bibliotecas de Classes Partilhadas (RCLs) permitiu reutilizar cerca de 90% do código de interface e lógica entre a aplicação móvel e a web, reduzindo drasticamente o tempo de desenvolvimento e a probabilidade de erros. O sistema é capaz de gerir todo o ciclo de vida de um produto, desde a submissão pelo fornecedor até à entrega da encomenda ao cliente.

Pontos Fortes:

- Segurança Robusta: Implementação correta de RBAC (Role-Based Access Control) e distinção clara entre dados públicos e privados.
- Experiência de Utilizador: A aplicação móvel oferece uma performance nativa fluida, superior a soluções baseadas apenas em webviews tradicionais.
- Escalabilidade: A separação entre API e Frontend permite que o sistema cresça facilmente (ex: criando um site público em Blazor WASM) sem alterar o backend.

Limitações e Trabalho Futuro:

- Pagamentos: O sistema atual simula o processo de pagamento. Uma evolução natural seria a integração com uma gateway real (ex: Stripe ou PayPal).
- Notificações: Embora funcional, a comunicação com o utilizador poderia ser melhorada através de notificações *push* na app ou envio de emails transacionais (confirmação de encomenda). Atualmente, o sistema utiliza um simulador de envio de emails (IdentityNoOpEmailSender).

9.2. Conclusão

O projeto MyCOLL representou um marco fundamental na nossa formação, permitindo aplicar conceitos avançados de sistemas distribuídos na orquestração técnica entre a API, o Backoffice e a aplicação móvel. A arquitetura baseada em .NET 8 e o uso estratégico de bibliotecas partilhadas (RCL) foram decisivos para superar a complexidade do desenvolvimento, garantindo uma solução final coesa, segura e de fácil manutenção.

O resultado obtido transcende o âmbito académico, constituindo uma base sólida para uma aplicação comercial escalável e adaptada às exigências do mercado real. Este trabalho consolidou competências essenciais em arquitetura de software e desenvolvimento *Full-Stack*, validando a criação de soluções modernas e multiplataforma com elevada partilha de código.